

Software maintenance

Software maintenance is the modification of software after delivery.

Software maintenance is often considered lower skilled and less rewarding than new development. As such, it is a common target for outsourcing or offshoring. Usually, the team developing the software is different from those who will be maintaining it. The developers lack an incentive to write the code to be easily maintained. Software is often delivered incomplete and almost always contains some bugs that the maintenance team must fix. Software maintenance often initially includes the development of new functionality, but as the product nears the end of its lifespan, maintenance is reduced to the bare minimum and then cut off entirely before the product is withdrawn.

Each maintenance cycle begins with a change request typically originating from an end user. That request is evaluated and if it is decided to implement it, the programmer studies the existing code to understand how it works before implementing the change. Testing to make sure the existing functionality is retained and the desired new functionality is added often comprises most of the maintenance cost.

Software maintenance is not as well studied as other phases of the software life cycle, despite comprising most of the cost. Understanding has not changed significantly since the 1980s. Software maintenance can be categorized into several types depending on whether it is preventative or reactive and whether it is seeking to add functionality or preserve existing functionality, the latter typically in the face of a changed environment.

History

In the early 1970s, companies began to separate out software maintenance with its own team of engineers to free up software development teams from support tasks.^[1] In 1972, R. G. Canning published "The Maintenance 'Iceberg'", in which he contended that software maintenance was an extension of software development with an additional input: the existing system.^[1] The discipline of software maintenance has changed little since then.^[2] One twenty-first century innovation has been companies deliberately releasing incomplete software and planning to finish it post-release. This type of change, and others that expand functionality, is often called software evolution instead of maintenance.^[2]

Software life cycle

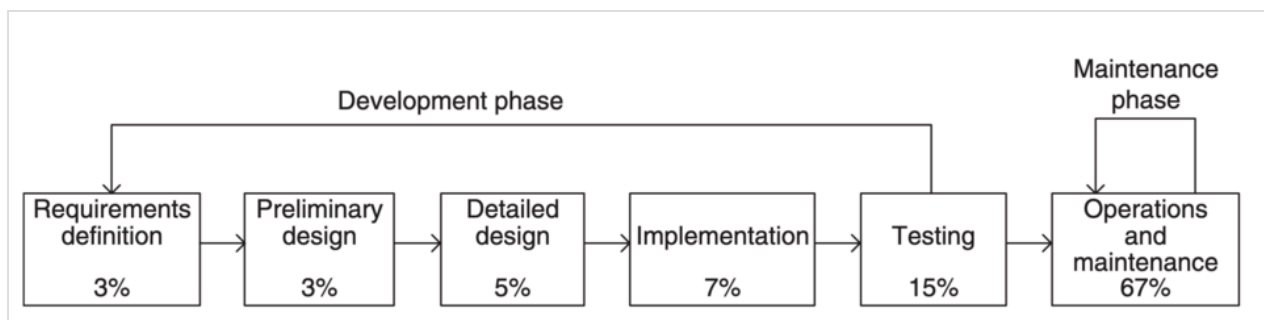


Diagram for a traditional software development life cycle from 1988

Despite testing and quality assurance, virtually all software contains bugs where the system does not work as intended. Post-release maintenance is necessary to remediate these bugs when they are found.^[3] Most software is a combination of pre-existing commercial off-the-shelf (COTS) and open-source software components with custom-written code. COTS and open-source software is typically updated over time, which can reduce the maintenance burden, but the modifications to these software components will need to be adjusted for in the final product.^[4] Unlike software development, which is focused on meeting specified requirements, software maintenance is driven by events—such as user requests or detection of a bug.^[5] Its main purpose is to preserve the usefulness of the software, usually in the face of changing requirements.^[6]

If conceived of as part of the software development life cycle, maintenance is the last and typically the longest phase of the cycle,^{[7][8]} comprising 80 to 90 percent of the lifecycle cost.^[9] Other models consider maintenance separate from software development, instead as part of the software maintenance life cycle (SMLC).^[8] SMLC models typically include understanding the code, modifying it, and revalidating it.^[8]

Transition from release to maintenance to end of the lifespan

Frequently, software is delivered in an incomplete state. Developers will test a product until running out of time or funding, because they face fewer consequences for an imperfect product than going over time or budget.^[10] The transition from the development to the maintenance team is often inefficient, without lists of known issues or validation tests, which the maintenance team will likely recreate.^[11] After release, members of the development team are likely to be reassigned or otherwise become unavailable. The maintenance team will require additional resources for the first year after release, both for technical support and fixing defects left over from development.^[10]

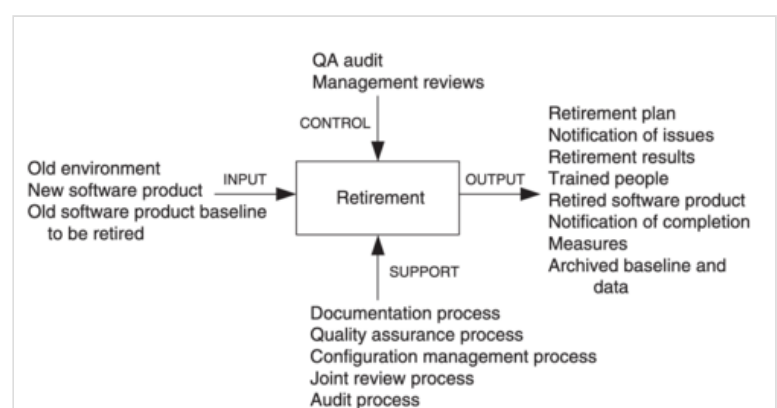


Diagram showing the steps for software retirement

Initially, software may go through a period of enhancements after release. New features are added according to user feedback. At some point, the company may decide that it is no longer profitable to make functional improvements, and restrict support to bug fixing and emergency updates. Changes become increasingly difficult and expensive due to lack of expertise or decaying architecture due to software aging. After a product is no longer maintained, and does not receive even this limited level of updating, some vendors will seek to extract revenue from the software as long as possible, even though the product is likely to become increasingly avoided. Eventually, the software will be withdrawn from the market, although it may remain in use. During this process, the software becomes a legacy system.^[12]

Change cycle

The first step in the change cycle is receiving a change request from a customer and analyzing it to confirm the problem and decide whether to implement the change.^[13] This may require input from multiple departments; for example, the marketing team can help evaluate whether the change is expected to bring more business.^[14] Software development effort estimation is a difficult problem, including for maintenance change requests,^[15] but the request is likely to be declined if it is too expensive or infeasible.^[16] If it is decided to implement the request, it can be assigned to a scheduled release and implemented.^[16] Although agile methodology does not have a maintenance phase,^[17] the change cycle can be enacted as a scrum sprint.^[18]

Understanding existing code is an essential step before modifying it.^[2] The rate of understanding depends both on the code base as well as the skill of the programmer.^[19] Following coding conventions such as using clear function and variable names that correspond to their purpose makes understanding easier.^[20] Use of conditional loop statements only if the code could execute more than once, and eliminating code that will never execute can also increase understandability.^[21] Experienced programmers have an easier time understanding what the code does at a high level.^[22] Software visualization is sometimes used to speed up this process.^[23]

Modification to the code may take place in any way. On the one hand, it is common to haphazardly apply a quick fix without being granted enough time to update the code documentation.^[24] On the other hand structured iterative enhancement can begin by changing the top-level requirements document and propagating the change down to lower levels of the system.^[25] Modification often includes code refactoring (improving the structure without changing functionality) and restructuring (improving structure and functionality at the same time).^[26] Unlike commercial software, free and open source software change cycles are largely restricted to coding and testing, with minimal documentation. Open-source software projects instead rely on mailing lists and a large number of contributors to understand the code base and fix bugs efficiently.^[27]

An additional problem with maintenance is that nearly every change to code will introduce new bugs or unexpected ripple effects, which require another round of fixes.^[2] Testing can consume most of the maintenance resource for safety-critical code, due to the need to revalidate the entire software if any changes are made.^[28] Revalidation may include code review, regression testing with a subset of unit tests, integration tests, and system tests.^[26] The goal of the testing is to verify that previous functionality is retained, and the new functionality has been added.^[29]

Categories of software maintenance

The key purpose of software maintenance is ensuring that the product continues to meet usability requirements. At times, this may mean extending the product's capabilities beyond what was initially envisioned.^[30]

According to the ISO/IEC 14764 specification, software maintenance can be classified into four types:^[31]

- Corrective maintenance: modification of software to fix a bug or other failure to meet requirements, typically reported by an end user.^{[31][32]}
- Preventive maintenance: forward-looking modification of software after delivery to ensure it continues to meet requirements or fix problems that have not manifested yet.^{[33][31]} This type of maintenance is performed especially on systems that are required to be highly safe or available.^[33] Software rejuvenation is one form of preventative maintenance to clean up state and prevent future problems.^[33]
- Adaptive maintenance: modification of software performed after delivery to ensure its continuing usability in a changed or changing environment.^{[31][33]}
- Perfective maintenance: enhancement of software after delivery to improve qualities such as user experience, processing efficiency, and maintainability.^{[33][34]} Perfective maintenance is necessary if other types of maintenance are carried out, because modification of an existing code base will otherwise increase complexity and cause the existing structure to deteriorate.^[34] Perfective maintenance may include rewriting documentation, code refactoring, and performance tuning.^[33]

According to some estimates, enhancement (the latter two categories) comprises some 80 percent of software maintenance.^[35]

Maintainability

Maintainability is the quality of software enabling it to be easily modified without breaking existing functionality.^[31] According to the ISO/IEC 14764 specification, activity to ensure software maintainability prior to release counts as part of software maintenance.^[5] Many software development organizations neglect maintainability, even though doing so will increase long-term costs.^[36] Technical debt is incurred when programmers, often out of laziness or urgency to meet a deadline, choose quick and dirty solutions rather than build maintainability into their code.^[37] A common cause is underestimates in software development effort estimation, leading to insufficient resources allocated to development.^[38] One important aspect is having a large amount of automated software tests that can detect if existing functionality is compromised by a change.^[31]

A maintainability index can be calculated with certain formulae from lines-of-code measures, McCabe measures and Halstead complexity measures.

The measurement and tracking of maintainability are intended to help reduce or reverse a system's tendency toward "code entropy" or degraded integrity, and to indicate when it becomes cheaper and/or less risky to rewrite the code than it is to change it.

A challenge with maintainability is that many software engineering courses do not emphasize it, and give out one-and-done assignments that have clear and unchanging specifications.^[39] Software engineering courses do not cover systems as complex as occur in the real world.^[40] Development engineers who know that they will not be responsible for maintaining the software do not have an incentive to build in maintainability.^[2]

Workforce

Maintenance is often considered an unrewarding job for software engineers, who, if assigned to maintenance, were more likely to quit.^{[41][42]} It often pays less than a comparable job in software development.^[42] The task is often assigned to temporary workers or lesser-skilled staff,^{[2][43]} although maintenance engineers are also typically older than developers, partly because they must be familiar with outdated technologies.^[43] In 2008, around 900,000 of the 1.3 million software engineers and programmers working in the United States were doing maintenance.^[44]

Companies started separate teams for maintenance, which led to outsourcing this work to a different company, and by the turn of the twenty-first century, sometimes offshoring the work to another country—whether as part of the original company or a separate entity.^{[45][9]} The typical sources of outsourcing are developed countries such as the United States, the United Kingdom, Japan, and Australia, while destinations are usually lower-cost countries such as China, India, Russia, and Ireland.^[46] Reasons for offshoring include taking advantage of lower labor costs, enabling around-the-clock support, reducing time pressure on developers, and to move support closer to the market for the product.^[47] Downsides of offshoring include communication barriers in the form of such factors as time zone and organizational disjunction and cultural differences.^[9] Despite many employers considering maintenance lower-skilled work and the phase of software development most suited to offshoring,^{[9][48]} it requires close communication with the customer and rapid response, both of which are hampered by these communication difficulties.^[9]

Alternatives to maintenance

In software engineering, the term legacy system does not have a fixed meaning, but often refers to older systems which are large, difficult to modify, and also necessary for current business needs. Often legacy systems are written in obsolete programming languages, lack documentation, have a deteriorating structure after years of changes, and depend on experts to keep it operational.^[49] When dealing with these systems, at some point so much technical debt accumulates that maintenance is not practical or economical.^[12] Other choices include:

- Freezing—do no more work on the legacy system.^[50] This option may be chosen if the vendor wants to continue to extract revenue as long as possible while avoiding maintenance costs.^[12]
- Outsourcing functionality of the legacy system to a different company, especially if it is not considered a core business function.^[50]
- Discarding the existing legacy system and redeveloping a new application from scratch to fulfill the same purpose as the legacy system.^[50] However, this approach is inefficient due to

discarding a working system, and with this approach there is a danger that the new system will not fulfill changing business requirements.^[50]

- Wrapping the legacy application in an abstraction layer to simplify outdated interfaces.^[50] The source code is not modified but the new interface allows a tried and tested component to be accessed by newer applications. This approach does not fix any of the issues with maintaining a legacy system.^[51] Databases, functions, and entire applications may be wrapped in this way.^[52]
- Migrating the legacy system to a new platform, which can reduce the expense of new software development by reusing the implementation, design, specification, and requirements of the legacy system.^[53] Migration can take 5 to 10 years, but results in greater flexibility and long-term savings in software maintenance.^[54] As much as 80 percent of the expense is in testing; that is, ensuring that the new system has the same output as the old system.^[55] After the new system is finished, there needs to be a transition from the old system to the new system with minimum disruption to business functions.^[55]

Research

Despite taking up the lion's share of software development resources, maintenance is the least studied phase of software development.^{[56][57]} Much of the literature has focused on how to develop maintainable code from the outset, with less focus on motivating engineers to make maintainability a priority.^[58] As of 2020, automated solutions for code refactoring to reduce maintenance effort are an active area of research,^[59] as is machine-learning enhanced maintainability assessment.^[60]

See also

- Cost estimation in software engineering
- Total cost of ownership, product cost including maintenance costs and other indirect costs

References

1. Tripathy & Naik 2014, p. 25.
2. Offutt, Jeff (January 2018). "Overview of Software Maintenance and Evolution" (<https://cs.gmu.edu/~offutt/classes/437/maintessays/maintEvolutionOverview.html>). George Mason University Department of Computer Science. Retrieved 5 May 2024.
3. Tripathy & Naik 2014, p. 4.
4. Tripathy & Naik 2014, pp. 5–6.
5. Tripathy & Naik 2014, p. 26.
6. Madhusudhan et al. 2017, p. 761.
7. Varga 2018, p. 3.
8. Tripathy & Naik 2014, p. 7.
9. Ulziit et al. 2015, p. 764.
10. Reifer 2012, p. 22.
11. Reifer 2012, p. 21.
12. Tripathy & Naik 2014, p. 89.
13. Madhusudhan et al. 2017, p. 763.
14. Tripathy & Naik 2014, p. 120.

15. Madhusudhan et al. 2017, p. 762.
16. Tripathy & Naik 2014, p. 123.
17. Ali et al. 2024, p. 126.
18. Ali et al. 2024, p. 130.
19. Tripathy & Naik 2014, p. 296.
20. Tripathy & Naik 2014, pp. 296–297.
21. Tripathy & Naik 2014, p. 309.
22. Tripathy & Naik 2014, p. 297.
23. Tripathy & Naik 2014, pp. 318–319.
24. Tripathy & Naik 2014, pp. 85–86.
25. Tripathy & Naik 2014, p. 86.
26. Tripathy & Naik 2014, p. 94.
27. Tripathy & Naik 2014, p. 59.
28. Reifer 2012, p. 5.
29. Tripathy & Naik 2014, p. 98.
30. Varga 2018, p. 4.
31. Varga 2018, p. 5.
32. Tripathy & Naik 2014, pp. 26–27.
33. Tripathy & Naik 2014, p. 27.
34. Varga 2018, pp. 5–6.
35. Varga 2018, p. 5 fn 4.
36. Varga 2018, p. 12.
37. Varga 2018, pp. 6–7.
38. Varga 2018, p. 7.
39. Varga 2018, pp. 7–8.
40. Varga 2018, p. 9.
41. Madhusudhan et al. 2017, p. 764.
42. Reifer 2012, p. 7.
43. Reifer 2012, p. 8.
44. Reifer 2012, p. 1.
45. Rahman et al. 2024, p. 1.
46. Rahman et al. 2021, Research Background.
47. Ulziit et al. 2015, p. 763.
48. Reifer 2012, p. 2.
49. Tripathy & Naik 2014, pp. 187–188.
50. Tripathy & Naik 2014, p. 188.
51. Tripathy & Naik 2014, p. 189.
52. Tripathy & Naik 2014, p. 191.
53. Tripathy & Naik 2014, pp. 188–189.
54. Tripathy & Naik 2014, p. 195.
55. Tripathy & Naik 2014, p. 196.
56. Madhusudhan et al. 2017, p. 759.
57. Ulziit et al. 2015, p. 766.
58. Reifer 2012, pp. 4–5.
59. Baqais & Alshayeb 2020, p. 459.

Sources

- Ali, Muhammad; Cheema, Sehrish Munawar; Naz, Ammerha; Pires, Ivan Miguel (2024). "SAMSEF: An Agile Software Maintenance Leveraging Scrum Framework for Improved Efficiency and Effectiveness". *Good Practices and New Perspectives in Information Systems and Technologies*. Lecture Notes in Networks and Systems. Vol. 989. Springer Nature Switzerland. pp. 126–136. doi:10.1007/978-3-031-60227-6_11 (https://doi.org/10.1007%2F978-3-031-60227-6_11). ISBN 978-3-031-60226-9.
- Alsolai, Hadeel; Roper, Marc (2020). "A systematic literature review of machine learning techniques for software maintainability prediction" (https://strathprints.strath.ac.uk/71689/1/A_Islolai_Roper_IST_2019_A_systematic_literature_review_of_machine_learning_techniques.pdf) (PDF). *Information and Software Technology*. **119** 106214. doi:10.1016/j.infsof.2019.106214 (<https://doi.org/10.1016%2Fj.infsof.2019.106214>).
- Baqais, Abdulrahman Ahmed Bobakr; Alshayeb, Mohammad (2020). "Automatic software refactoring: a systematic literature review". *Software Quality Journal*. **28** (2): 459–502. doi:10.1007/s11219-019-09477-y (<https://doi.org/10.1007%2Fs11219-019-09477-y>).
- Madhusudhan, V.; Suma, V.; Rao, Jawahar J. (2017). *Software Maintenance: From the Perspective of Effort and Cost Requirement*. Proceedings of the International Conference on Data Engineering and Communication Technology. Springer. pp. 759–768. ISBN 978-981-10-1678-3.
- Rahman, Hanif Ur; da Silva, Alberto Rodrigues; Alzayed, Asaad; Raza, Mushtaq (2024). "A Systematic Literature Review on Software Maintenance Offshoring Decisions". *Information and Software Technology*. **172** 107475. doi:10.1016/j.infsof.2024.107475 (<https://doi.org/10.1016%2Fj.infsof.2024.107475>).
- Rahman, Hanif Ur; Raza, Mushtaq; Afsar, Palwasha; Khan, Habib Ullah (2021). "Empirical Investigation of Influencing Factors Regarding Offshore Outsourcing Decision of Application Maintenance" (<https://doi.org/10.1109%2FACCESS.2021.3073315>). *IEEE Access*. **9**: 58589–58608. Bibcode:2021IEEEA...958589R (<https://ui.adsabs.harvard.edu/abs/2021IEEEA...958589R>). doi:10.1109/ACCESS.2021.3073315 (<https://doi.org/10.1109%2FACCESS.2021.3073315>). hdl:10576/37687 (<https://hdl.handle.net/10576%2F37687>). ISSN 2169-3536 (<https://search.worldcat.org/issn/2169-3536>).
- Reifer, Donald J. (2012). *Software Maintenance Success Recipes*. CRC Press. ISBN 978-1-4398-5167-8.
- Tripathy, Priyadarshi; Naik, Kshirasagar (2014). *Software Evolution and Maintenance: A Practitioner's Approach*. John Wiley & Sons. ISBN 978-0-470-60341-3.
- Ulziit, Bayarbuyan; Warraich, Zeeshan Akhtar; Gencel, Cigdem; Petersen, Kai (2015). "A conceptual framework of challenges and solutions for managing global software maintenance". *Journal of Software: Evolution and Process*. **27** (10): 763–792. doi:10.1002/smr.1720 (<https://doi.org/10.1002%2Fsmr.1720>).
- Varga, Ervin (2018). *Unraveling Software Maintenance and Evolution: Thinking Outside the Box*. Springer. ISBN 978-3-319-71303-8.